



Documentazione Completa: Parsing e Validazione Mappa Cub3D



Indice

- [Panoramica del Sistema](#)
 - [Strutture Dati](#)
 - [Flusso di Parsing Completo](#)
 - [Fase 1: Parsing delle Impostazioni](#)
 - [Fase 2: Parsing della Mappa](#)
 - [Fase 3: Validazione della Mappa](#)
 - [Edge Cases e Gestione Errori](#)
 - [Esempi Pratici](#)
 - [Riferimento Messaggi di Errore](#)
-



Panoramica del Sistema

Il sistema di parsing e validazione della mappa Cub3D è diviso in **tre fasi principali**:

1. **Parsing delle Impostazioni** (texture + colori)
2. **Parsing della Mappa** (lettura e normalizzazione)
3. **Validazione della Mappa** (controlli di integrità)

File Coinvolti

```
src/parsing/
├─ input.c           # Parsing impostazioni e coordinamento
├─ parse_settings.c  # Parser texture (NO, SO, WE, EA)
├─ parse_color.c     # Parser colori (F, C)
├─ parse_texture.c   # Validazione path texture
├─ parse_helpers.c   # Funzioni utility
├─ map.c             # Entry point parsing mappa
├─ map_read.c        # Lettura righe mappa
├─ map_validate.c    # Validazione alto livello
└─ map_check.c       # Controllo muri e vicini
```



Strutture Dati

t_input - Impostazioni di Configurazione

```
typedef struct s_input
{
    char    *path_no;      // Path texture Nord
    char    *path_so;      // Path texture Sud
    char    *path_we;      // Path texture Ovest
    char    *path_ea;      // Path texture Est
    t_color floor;         // Colore pavimento (RGB)
    t_color ceiling;       // Colore soffitto (RGB)
    int     has_no;        // Flag: texture Nord trovata
    int     has_so;        // Flag: texture Sud trovata
    int     has_we;        // Flag: texture Ovest trovata
    int     has_ea;        // Flag: texture Est trovata
    int     has_floor;     // Flag: colore pavimento trovato
    int     has_ceiling;   // Flag: colore soffitto trovato
} t_input;
```

t_color - Colore RGB

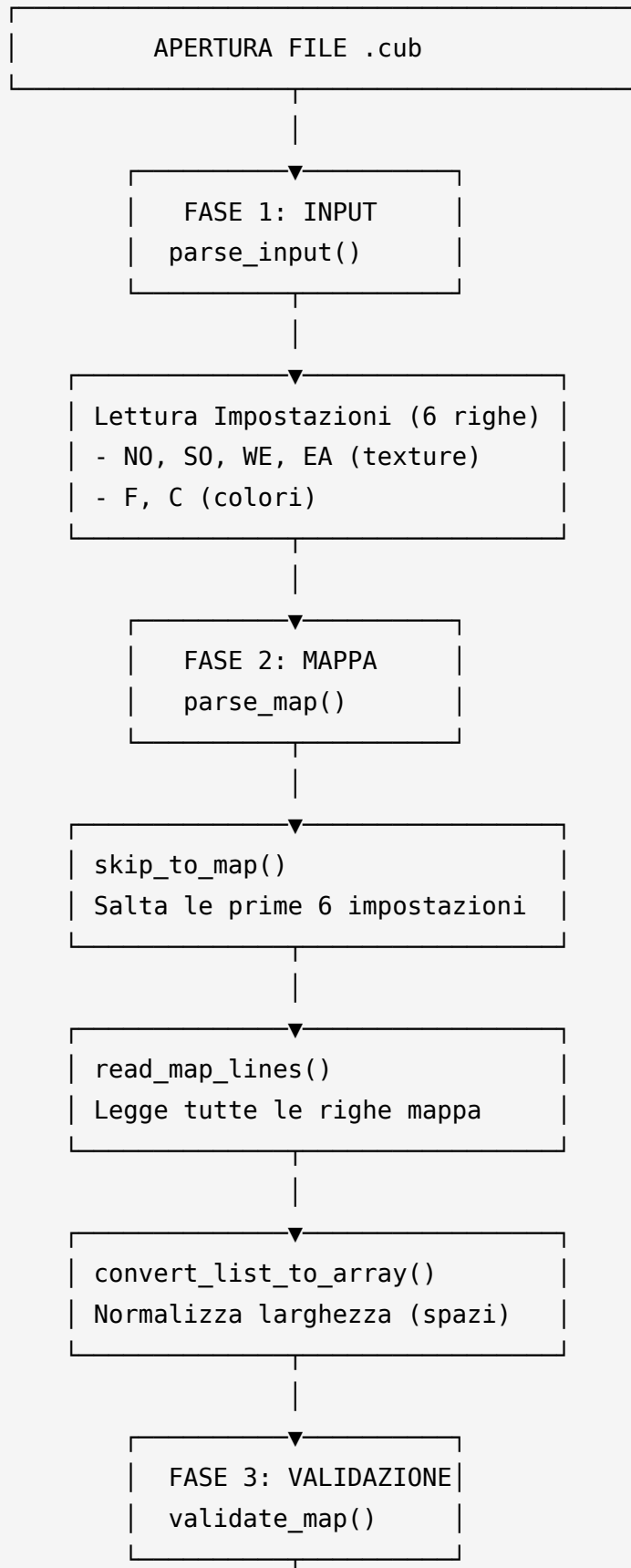
```
typedef struct s_color
{
    int r; // Rosso [0-255]
    int g; // Verde [0-255]
    int b; // Blu [0-255]
} t_color;
```

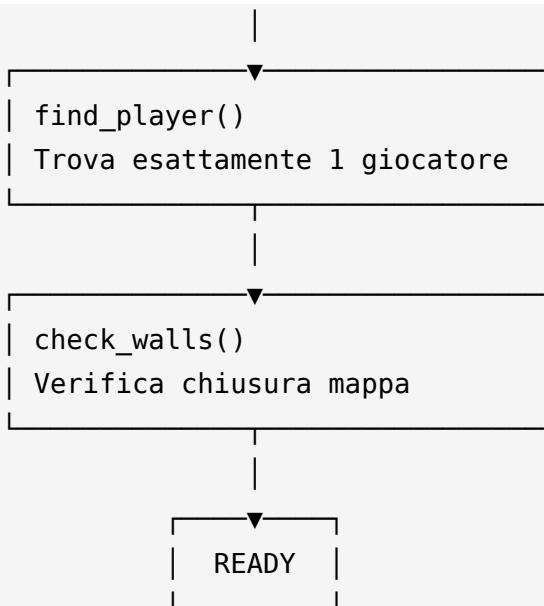
t_map - Dati della Mappa

```
typedef struct s_map
{
    int width; // Larghezza massima (normalizzata)
    int height; // Altezza (numero di righe)
    char **map; // Array 2D della mappa
    int player_x; // Posizione X del giocatore
    int player_y; // Posizione Y del giocatore
    char player_dir; // Direzione: N, S, E, W
} t_map;
```



Flusso di Parsing Completo





FASE 1: Parsing delle Impostazioni

Funzione: `parse_input()` - Entry Point

File: `input.c`

```
int parse_input(char *file_path, t_input *input)
```

Processo di Lettura

1. Inizializza la struttura `t_input` (tutti i flag a 0)
2. Legge il file riga per riga
3. Per ogni riga:
 - Salta righe vuote (solo spazi/tab/newline)
 - Prova a parsare come impostazione
 - Se trova tutte le 6 impostazioni + riga mappa → fine
4. Verifica che tutte le 6 impostazioni siano state trovate



Parsing Texture (NO, SO, WE, EA)

File: `parse_settings.c` , `parse_texture.c`

Formato Accettato

```
NO path/to/north_texture.xpm
SO path/to/south_texture.xpm
WE path/to/west_texture.xpm
EA path/to/east_texture.xpm
```

Validazioni

-  Path non vuoto
-  File esistente (verificato con `open()`)
-  Whitespace ignorato prima e dopo il path
-  Duplicati non ammessi
-  File inesistente

Funzione Chiave: `parse_texture_path()`

```
char *parse_texture_path(char *line)
{
    // 1. Rimuove whitespace iniziale
    // 2. Estrae path fino a spazio/tab/newline
    // 3. Verifica che il file esista (open)
    // 4. Restituisce path allocato
}
```

Errori Possibili:

- `"Invalid texture path\n"` - path vuoto
- `"Texture file not found\n"` - file inesistente
- `"Duplicate XX texture\n"` - texture già specificata



Parsing Colori (F, C)

File: `parse_color.c`

Formato Accettato

```
F 220,100,0      # Floor (pavimento)
C 225,30,0       # Ceiling (soffitto)
```

Validazioni

-  Esattamente 3 valori separati da virgola
-  Ogni valore è un numero valido
-  Range: $0 \leq \text{valore} \leq 255$
-  Whitespace ignorato
-  Duplicati non ammessi
-  Valori < 0 o > 255
-  Caratteri non numerici

Processo di Parsing

```
int parse_color(char *line, t_color *color)
{
    // 1. Split per virgola → array di 3 stringhe
    // 2. Verifica che ci siano esattamente 3 elementi
    // 3. Per ogni elemento:
    //     - Verifica che sia numerico
    //     - Converte con ft_atoi()
    //     - Verifica range [0, 255]
    // 4. Assegna a color->r, color->g, color->b
}
```

Errori Possibili:

- "Color must have exactly 3 RGB values\n" - numero elementi $\neq 3$
- "Invalid RGB format\n" - caratteri non numerici
- "RGB values must be in range [0, 255]\n" - fuori range
- "Duplicate floor color\n" / "Duplicate ceiling color\n"

✓ Controllo Completezza

Funzione: `all_settings_found()`

Verifica che **tutte e 6** le impostazioni siano state trovate:

```
return (input->has_no && input->has_so && input->has_we  
        && input->has_ea && input->has_floor && input->has_ceiling);
```

Errore:

- `"Missing required settings\n"` - una o più impostazioni mancanti

FASE 2: Parsing della Mappa

Funzione: `parse_map()` - Entry Point

File: `map.c`

```
int parse_map(char *file_path, t_map *map)
```

Processo

1. **Skip Impostazioni** - `skip_to_map()`
2. **Lettura Righe** - `read_map_lines()`
3. **Calcolo Dimensioni** - `init_map_struct()`
4. **Normalizzazione** - `convert_list_to_array()`
5. **Validazione** - `validate_map()`

1 Skip delle Impostazioni

Funzione: `skip_to_map()` - File: `map_read.c`

```
void skip_to_map(int fd)
{
    // Legge righe finché non trova 6 impostazioni (N0/S0/WE/EA/F/C)
    // Lascia il file descriptor posizionato all'inizio della mappa
}
```

2 Lettura Righe della Mappa

Funzione: `read_map_lines()` - File: `map_read.c`

```
t_list *read_map_lines(int fd)
```

Logica

- Legge righe finché `get_next_line()` restituisce NULL
- Accetta una riga se `is_map_line()` restituisce true
- Si ferma quando trova una riga non-mappa **dopo** aver iniziato
- Crea una linked list (`t_list`) con le righe

Identificazione Riga Mappa

Funzione: `is_map_line()` - File: `parse_helpers.c`

Una riga è valida se:

-  Contiene **solo** caratteri: `'0'`, `'1'`, `' '`, `'\t'`, `'N'`, `'S'`, `'E'`, `'W'`
-  Contiene almeno un carattere non-whitespace

```
int is_map_line(char *line)
{
    // Per ogni carattere (fino a \n):
    //   - Se non è map_char → return 0
    //   - Se non è spazio/tab → has_content = 1
    // return has_content
}
```

Esempio:

"1111111\n"	→ ✓ valida
"10001\n"	→ ✓ valida
" \t \n"	→ ✗ solo whitespace
"1001X1\n"	→ ✗ carattere 'X' invalido

3 Calcolo Dimensioni

Funzione: `init_map_struct()` - File: `map_read.c`

```
void init_map_struct(t_map *map, t_list *map_lines)
{
    map->height = ft_lstsize(map_lines);    // Numero di righe
    map->width = get_max_width(map_lines);  // Larghezza massima
    map->player_x = -1;
    map->player_y = -1;
    map->player_dir = '\0';
}
```

Funzione: `get_max_width()`

- Trova la riga più lunga
- Esclude il carattere `\n` dalla lunghezza

4 Normalizzazione - Array 2D

Funzione: `convert_list_to_array()` - File: `map.c`

Processo

Converte la linked list in un array 2D **normalizzato**:

- Tutte le righe hanno la **stessa larghezza** (= `map->width`)
- Righe più corte vengono **riempite con spazi** a destra
- Array termina con `NULL`

Funzione Chiave: `fill_line_with_spaces()`

```
static void fill_line_with_spaces(char *dest, char *src, int width)
{
    // 1. Copia caratteri da src (escluso \n)
    // 2. Riempie il resto con ' ' fino a width
    // 3. Termina con \0
}
```

Esempio di Normalizzazione:

Input (lunghezze variabili):

```
111111
100001
1000000001
111111
```

Output (larghezza = 10):

```
"111111    " (10 caratteri)
"100001    " (10 caratteri)
"1000000001" (10 caratteri)
"111111    " (10 caratteri)
```

Errore:

- "Memory allocation failed\n" - malloc fallito

FASE 3: Validazione della Mappa

Funzione: `validate_map()` - Entry Point

File: `map_validate.c`

```

int validate_map(t_map *map)
{
    if (find_player(map))    // 1. Trova giocatore
        return (1);
    if (check_walls(map))    // 2. Verifica chiusura
        return (1);
    return (0);
}

```

1 Validazione Giocatore

Funzione: `find_player()` - File: `map_validate.c`

Obiettivi

1. Trovare **esattamente 1** giocatore (N/S/E/W)
2. Salvare posizione e direzione

Algoritmo

```

int find_player(t_map *map)
{
    count = 0;
    for (y = 0; y < height; y++)
        for (x = 0; x < width; x++)
            if (is_player_char(map[y][x]))
            {
                map->player_x = x;
                map->player_y = y;
                map->player_dir = map[y][x];
                count++;
            }

    if (count != 1)
        return ERROR;
    return OK;
}

```

Funzione: `is_player_char()`

```
int is_player_char(char c)
{
    return (c == 'N' || c == 'S' || c == 'E' || c == 'W');
}
```

Direzioni:

- `'N'` → Nord (guarda verso l'alto)
- `'S'` → Sud (guarda verso il basso)
- `'E'` → Est (guarda verso destra)
- `'W'` → Ovest (guarda verso sinistra)

Errore:

- `"Map must have exactly one player\n"` - 0 giocatori o > 1

2 Validazione Chiusura Muri

Funzione: `check_walls()` - File: `map_check.c`

Obiettivo

Verificare che **ogni spazio percorribile** sia completamente circondato da muri.

Algoritmo di Alto Livello

```
int check_walls(t_map *map)
{
    for (y = 0; y < height; y++)
        for (x = 0; x < width; x++)
            if (check_position(map, y, x)) // ← controlla posizione
                return ERROR;
    return OK;
}
```



Controllo Singola Posizione

Funzione: `check_position()` - File: `map_check.c`

```
int check_position(t_map *map, int y, int x)
{
    // 1. FILTRO: ignora celle non percorribili
    if (map->map[y][x] != '0' && !is_player_char(map->map[y][x]))
        return (0); // Non è spazio percorribile → skip

    // 2. CONTROLLO BORDI: spazi percorribili sui bordi = ERRORE
    if (y == 0 || y == map->height - 1 ||
        x == 0 || x == map->width - 1)
        return ERROR("Map not surrounded by walls");

    // 3. CONTROLLO 4 VICINI
    if (check_neighbor(map, y - 1, x)) // Sopra
        return (1);
    if (check_neighbor(map, y + 1, x)) // Sotto
        return (1);
    if (check_neighbor(map, y, x - 1)) // Sinistra
        return (1);
    if (check_neighbor(map, y, x + 1)) // Destra
        return (1);

    return (0); // OK
}
```

Fase 1: Filtro Celle

Controlla **solo** celle percorribili:

- `'0'` → spazio vuoto percorribile
- `'N'`, `'S'`, `'E'`, `'W'` → posizione giocatore (percorribile)
- `'1'` → muro (ignorato)
- `' '` → spazio esterno (ignorato)

Fase 2: Controllo Bordi

Se una cella percorribile si trova sul **bordo esterno** della mappa:

0 ← ERRORE!	y == 0 (bordo superiore)
-------------	--------------------------

Perché? La mappa deve essere **chiusa**: non si può uscire.

Fase 3: Controllo Vicini

Per ogni vicino (sopra/sotto/sinistra/destra), chiama `check_neighbor()`.

Controllo Singolo Vicino

Funzione: `check_neighbor()` - File: `map_check.c`

```
int check_neighbor(t_map *map, int y, int x)
{
    if (!is_valid_neighbor(map, y, x) &&           // NON è un muro valido
        map->map[y][x] != '0' &&                   // NON è spazio '0'
        !is_player_char(map->map[y][x]))           // NON è giocatore
    {
        return ERROR("Map not surrounded by walls");
    }
    return OK;
}
```

Logica AND a 3 Condizioni

L'errore viene restituito **solo se tutte e 3** le condizioni sono vere:

1. `!is_valid_neighbor()` → Il vicino NON è un muro '1' valido
2. `!= '0'` → Il vicino NON è uno spazio percorribile '0'
3. `!is_player_char()` → Il vicino NON è il giocatore

In altre parole: Errore se il vicino è uno **spazio esterno** o un **carattere**

invalido.

Validazione Vicino

Funzione: `is_valid_neighbor()` - File: `map_check.c`

```
int is_valid_neighbor(t_map *map, int y, int x)
{
    // 1. Controllo limiti
    if (y < 0 || y >= map->height || x < 0 || x >= map->width)
        return (0); // Fuori dalla mappa

    // 2. Controllo muro
    return (map->map[y][x] == '1'); // 1 se è muro, 0 altrimenti
}
```

Restituisce 1 (true) solo se:

- Coordinate dentro i limiti **E**
- Cella contiene muro `'1'`

Analisi Completa dei Casi

Tabella Decisionale: Controllo Vicino

Contenuto Vicino	<code>is_valid_neighbor()</code>	Cond. 1	Cond. 2	Cond. 3	Risultato	Motivaz
<code>'1'</code> (muro)	1 (true)	0 ❌	-	-	✅ OK	Confine sicuro
<code>'0'</code> (spazio)	0 (false)	1 ✅	0 ❌	-	✅ OK	Spazio adiacento (sarà controllato)

Contenuto Vicino	<code>is_valid_neighbor()</code>	Cond. 1	Cond. 2	Cond. 3	Risultato	Motivaz
'N/S/E/W' (player)	0 (false)	1	1	0	OK	Player = spazio percorrib
'.' (esterno)	0 (false)	1	1	1	ERRORE	Mappa non chiusa!
Altri caratteri	0 (false)	1	1	1	ERRORE	Carattere invalido
Fuori limiti	0 (false)	1	1	1	ERRORE	(già bloccato prima)

Esempio di Analisi: Mappa Valid

```
111111
10N001
100001
111111
```

Controllo cella (1, 1) con '0':

1. Non è bordo: $y=1, x=1 \rightarrow \text{OK}$
2. Vicino sopra (0, 1): '1' $\rightarrow$ `is_valid_neighbor()` = 1 $\rightarrow$ **OK**
3. Vicino sotto (2, 1): '0' $\rightarrow$ cond. 2 falsa $\rightarrow$ **OK**
4. Vicino sinistra (1, 0): '1' $\rightarrow$ `is_valid_neighbor()` = 1 $\rightarrow$ **OK**
5. Vicino destra (1, 2): 'N' $\rightarrow$ cond. 3 falsa $\rightarrow$ **OK**

Risultato: Cella valida

Esempio di Analisi: Mappa NON Valida

```
111111
10N0 1  ← Spazio ' ' tra 0 e 1
100001
111111
```

Controllo cella (1, 3) con '0':

1. Non è bordo: $y=1, x=3 \rightarrow$ OK
2. Vicino sopra (0, 3): '1' $\rightarrow$ **OK** ✓
3. Vicino sotto (2, 3): '0' $\rightarrow$ **OK** ✓
4. Vicino sinistra (1, 2): 'N' $\rightarrow$ **OK** ✓
5. Vicino destra (1, 4): ' ' (spazio esterno)
 - `is_valid_neighbor()` = 0
 - `!= '0'` $\rightarrow$ true (è ' ')
 - `!is_player_char()` $\rightarrow$ true
 - **Tutte e 3 condizioni vere $\rightarrow$ ✗ ERRORE!**

Messaggio: "Map not surrounded by walls\n"

Edge Cases e Gestione Errori

1. Mappa Vuota

```
N0 ./texture.xpm
S0 ./texture.xpm
WE ./texture.xpm
EA ./texture.xpm
F 220,100,0
C 225,30,0

(nessuna mappa)
```

Errore: "No map found in file\n"

2. Riga Vuota nel Mezzo della Mappa

```
1111
1001

1001 ← Riga vuota prima
1111
```

Comportamento: La mappa si ferma alla riga vuota (legge solo le prime 2 righe)

3. Mappa con Buchi (Flood Fill Case)

```
11111
10001
10 01 ← Spazio ' ' interno
10001
11111
```

Errore: "Map not surrounded by walls\n"

- Quando controlla '0' in (2, 1), trova spazio ' ' come vicino

4. Giocatore sul Bordo

```
N1111
10001
11111
```

Errore: "Map not surrounded by walls\n"

- `check_position()` rileva che 'N' ha $y=0$ (bordo)

5. Multipli Giocatori

```
11111  
1N0E1  
11111
```

Errore: "Map must have exactly one player\n"

- `find_player()` trova count=2

6. Nessun Giocatore

```
11111  
10001  
11111
```

Errore: "Map must have exactly one player\n"

- `find_player()` trova count=0

7. Colore Fuori Range

```
F 300,100,50
```

Errore: "RGB values must be in range [0, 255]\n"

8. Colore con 2 Valori

```
F 220,100
```

Errore: "Color must have exactly 3 RGB values\n"

9. Texture Duplicata

```
NO ./north.xpm
NO ./another_north.xpm
```

Errore: "Duplicate NO texture\n"

10. Texture Inesistente

```
NO ./non_existent.xpm
```

Errore: "Texture file not found\n"

11. Mappa con Carattere Invalido

```
11111
10X01
11111
```

Comportamento: `is_map_line()` restituisce false per la riga → mappa si ferma prima

12. Spazi alla Fine delle Righe (Normalizzazione)

```
111
1001
11111111
```

Comportamento: Normalizzato a larghezza 8:

```
"111      " (riempito con spazi)
"1001     " (già 8 caratteri)
"11111111" (già 8 caratteri)
```

13. Impostazioni in Ordine Casuale

```
C 225,30,0
NO ./north.xpm
F 220,100,0
EA ./east.xpm
SO ./south.xpm
WE ./west.xpm
```

Comportamento:  **OK** - l'ordine non importa, basta che ci siano tutte

14. Righe Vuote tra Impostazioni

```
NO ./north.xpm

SO ./south.xpm

WE ./west.xpm
```

Comportamento:  **OK** - righe vuote ignorate

15. Mappa Non Rettangolare (Prima della Normalizzazione)

```
111111
10001
1000001
111
```

Comportamento: Normalizzata a larghezza 7:

```
"111111 "
"10001  "
"1000001"
"111    "
```

Se la riga più corta crea buchi → errore durante `check_walls()`

16. Mappa con Solo Muri

```
111111
111111
111111
```

Errore: "Map must have exactly one player\n"

- Nessun giocatore trovato

17. Tab e Spazi Misti

```
N0\t./north.xpm
F  220,  100  ,  0
```

Comportamento:  **OK** - `skip_whitespace()` gestisce sia tab che spazi

Riferimento Messaggi di Errore

Errori File

Messaggio	Causa	File
"Failed to open file\n"	File .cub non trovato	input.c , map.c

Errori Impostazioni

Messaggio	Causa	File
"Missing required settings\n"	Mancano una o più delle 6 impostazioni	input.c

Messaggio	Causa	File
"Invalid line or incomplete settings\n"	Riga non riconosciuta prima di avere tutte le impostazioni	input.c
"Duplicate XX texture\n"	Texture specificata più volte	parse_settings.c
"Invalid texture path\n"	Path texture vuoto	parse_texture.c
"Texture file not found\n"	File texture non esiste	parse_texture.c
"Duplicate floor color\n"	Colore pavimento specificato più volte	parse_settings.c
"Duplicate ceiling color\n"	Colore soffitto specificato più volte	parse_settings.c
"Color must have exactly 3 RGB values\n"	Formato colore non R,G,B	parse_color.c
"Invalid RGB format\n"	Valori RGB non numerici	parse_color.c
"RGB values must be in range [0, 255]\n"	Valore RGB < 0 o > 255	parse_color.c

Errori Mappa

Messaggio	Causa	File
"No map found in file\n"	Nessuna riga mappa nel file	map.c
"Memory allocation failed\n"	malloc() fallito	map.c
"Map must have exactly one player\n"	0 giocatori o più di 1	map_validate.c
"Map not surrounded by walls\n"	Spazio percorribile non chiuso o sul bordo	map_check.c



Esempi Pratici



Esempio 1: File Valido Completo

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
F 220,100,0
C 225,30,0
```

```
1111111111
1000000001
1000N00001
1000000001
1111111111
```

Risultato:

- Tutte le 6 impostazioni presenti

- Giocatore 'N' in posizione (4, 2) ✓
- Tutti gli spazi percorribili circondati da '1' ✓
- **MAPPA VALIDA** ✓

✓ Esempio 2: Mappa con Whitespace

```

N0 ./textures/north.xpm
S0 ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
F 220,100,0
C 225,30,0

```

```

1111111111
1000000001
1000500001
1000000001
1111111111

```

Risultato:

- Whitespace iniziale nelle righe mappa è parte della mappa (normalizzato)
- Se gli spazi esterni formano un "buco", darà errore
- **Dipende dalla struttura degli spazi** ⚠

✗ Esempio 3: Mappa Non Chiusa

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
F 220,100,0
C 225,30,0

11111
10001
 00E1 ← Spazio all'inizio
10001
11111
```

Risultato:

- Normalizzato: prima cella di riga 2 è ' '
- '0' in (1, 1) ha vicino sinistro ' ' (spazio esterno)
- **ERRORE:** "Map not surrounded by walls\n" ✗

✗ Esempio 4: Multipli Giocatori

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
F 220,100,0
C 225,30,0

1111111
1N000E1
1111111
```

Risultato:

- find_player() trova 2 giocatori (N ed E)
- **ERRORE:** "Map must have exactly one player\n" ✗

✗ Esempio 5: Colore Invalido

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
F 220,100,INVALID
C 225,30,0
```

```
1111111
10N0001
1111111
```

Risultato:

- `parse_color()` fallisce su "INVALID"
- **ERRORE:** "Invalid RGB format\n" ✗

🎓 Best Practices e Note Implementative

1. Memoria

- Tutte le stringhe allocate vengono liberate in caso di errore
- Le funzioni di parsing gestiscono la loro memoria internamente
- La mappa normalizzata richiede: `height * (width + 1) * sizeof(char)`

2. Performance

- La validazione è $O(\text{width} \times \text{height})$ per ogni cella
- Il parsing delle impostazioni è $O(n)$ con n = numero righe

3. Robustezza

- Tutti i file vengono verificati con `open()` prima dell'uso
- I valori RGB sono controllati prima della conversione

- La mappa viene normalizzata per gestire righe di lunghezza variabile

4. Estensibilità

- Aggiungere nuove impostazioni: modificare `all_settings_found()`
 - Aggiungere nuovi caratteri mappa: modificare `is_map_char()`
 - Aggiungere validazioni: estendere `validate_map()`
-



Statistiche e Limiti

Limiti Supportati

- **Dimensioni mappa:** Limitate solo dalla memoria disponibile
- **Range RGB:** 0-255 (standard)
- **Numero texture:** 4 (NO, SO, WE, EA)
- **Numero giocatori:** Esattamente 1
- **Caratteri mappa:** `'0'`, `'1'`, `' '`, `'N'`, `'S'`, `'E'`, `'W'`

Complessità

- **Parsing impostazioni:** $O(n)$ - n = numero righe file
 - **Parsing mappa:** $O(m)$ - m = numero righe mappa
 - **Validazione:** $O(w \times h)$ - w = larghezza, h = altezza
 - **Memoria:** $O(w \times h)$ - per array 2D normalizzato
-



Conclusione

Questo sistema di parsing e validazione garantisce che:

1. ☒ Tutte le impostazioni necessarie sono presenti e valide
2. ☒ La mappa è ben formata (solo caratteri validi)
3. ☒ Esiste esattamente un giocatore

4. ☒ La mappa è completamente chiusa (flood-fill proof)
5. ☒ Nessuno spazio percorribile è esposto all'esterno

Il sistema è **robusto, modulare** e gestisce correttamente tutti gli edge case comuni.

Documentazione generata per: Cub3D Project

Data: Ottobre 2025

Autore: Sistema di Parsing Mappa